rsna-intracranial-aneurysm-detection

7th place solution - 3D nnU-Net + blob regression (again)

Rank	7
Team	MIC-DKFZ
Author	Stefan Denner
Topic ID	612039
Version	Original

Thanks to @evancalabrese, @ryanholbrook, RSNA, and Kaggle for organizing this Intracranial Aneurysm Detection competition!

Overview (TLDR)

Here's a brief rundown of our solution — it's straightforward and easy to implement:

- We formulate the task as multichannel blob regression, optimized using a TopK (20%) BCE loss and then taking the maximum per channel as probability prediction.
- We build on nnU-Net, the leading framework for 3D medical image segmentation. We already adapted it for our 2nd place solution in the BYU - Locating Bacterial Flagellar Motors 2025.
- Our model is a 3D U-Net with a residual encoder, trained from scratch.
- Inference is done with a single model without test-time augmentation (due to time restrictions).
- Our model achieved a score of 0.83 / 0.83 on the public/private leaderboard.

Who are we?

We are a team of colleagues (scientists and PhD students) affiliated with the Divisions of Medical Image Computing at the German Cancer Research Center, as well as Helmholtz Imaging. Our expertise lies in 3D image analysis — particularly in solving 3D segmentation problems and developing infrastructure to bring algorithms into clinical practice.

Method

We modeled the task as a heatmap regression and built up on our 2nd place solution in the BYU - Locating Bacterial Flagellar Motors 2025.

Data used

We converted the DICOM series to .nii.gz format using the pydicom library, processing series with 2D DICOM files in parallel. We also used this library to extract information on spacing, origin, and direction. We then converted the resulting image into a SimpleITK image and oriented it in RAS orientation.

To reduce computing resources, we derived a [200, 160, 160] mm cubic Region of Interest (ROI) on the central superior region of the image. We ensured that all aneurysms in the training set were included in the ROI. Fig. 1 depicts the ROI on top of the image.

We observed that several series presented defects such as unexpected orientations, empty series, shunt artifacts, movement artifacts, or images with an empty superior space. We decided to keep those series with mild artifacts such as mis-orientations and fixed them via flipping. Series with stronger artifacts such as totally empty images were discarded. In total, ten volumes were discarded.

Figure 1. Axial, coronal and sagittal views of example image series and the ROI box, in red.

Preprocessing

Data preprocessing was completed with the self-configurable segmentation framework nnU-Net, following a 3D full resolution configuration. The volumes were loaded with a SimpleITK reader that also tries to enforce RAS orientation. All images were resampled to the median spacing found from all training images: 0.70, 0.47, 0.47, being normalized via z-score normalization based on a global mean and a global standard deviation extracted from the training dataset.

The images were resampled with a special resampler from PyTorch, which is faster than other commonly used functions such as scipy.ndimage.zoom, given the strong time constraints.

Network architecture

We use nnU-Net's ResEnc, which is essentially a UNet with a residual encoder and a lightweight convolutional decoder. The architecture included six stages with [32, 64, 128, 256, 320, 320] features in each stage, respectively.

Training Procedure

We split the provided challenge data into five cross-validation folds, stratifying for modalities across folds, rather than on vessel classes, since we wanted to ensure an adequate performance across all image modalities. Since we joined the challenge relatively late and there were many potential design choices to test, most of the hyperparameter tuning happened exclusively on the first fold of the cross-validation scheme.

Blob Regression with nnU-Net

nnU-Net is built for semantic segmentation. This also includes its expected data structure. To make it compatible with aneurysm regression we store the ground truth as semantic segmentation maps, where each aneurysm is encoded with a sphere (r=5 voxels) with an integer label representing the vessel class of the ground-truth aneurysm. These spheres are treated by nnU-Net as segmentations and are passed through the data loading and augmentation pipeline as nnU-Net normally would, thus properly applying rotations, mirroring etc, although we did not apply mirroring augmentations in the left/right axis, since several of the labels contained a left/right codification. At the end of the dataloading pipeline we inject a custom transform

that converts each aneurysm instance into a blob of the respective channel. We model the 14 classes as separate channels, where the 14th class (Aneurysm Present) is the pixelwise maximum of the 13 anatomical classes.

We use ‘EDT blobs’, basically 3D spheres that were transformed using the Euclidean Distance Transform (EDT) and rescaled to have a value range of [0, 1]. The optimized sphere size in the first cross-validation fold was 65 voxels. We experimented with sphere sizes from 15 to 95 voxel radii.

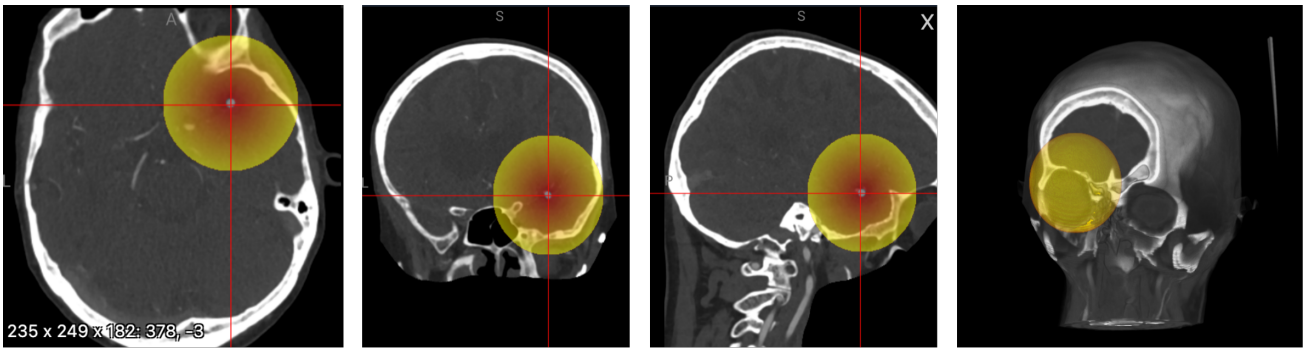

Figure 2: Blob (EDT, radius 65) at the Right Middle Cerebral Artery (Image not resampled yet)

Hyperparameters

Our final model was trained with a batch size of 32 and a patch size of 96,x160x128 voxels. Initial learning rate is 0.01 and is decayed over the course of the training using polyLR schedule (same as default nnU-Net). We train with SGD for 3000 epochs (250 iterations per epoch). The loss function utilized is binary cross-entropy, computed only on the 20% worst voxels (the ones with the highest loss value, computed over the entire batch).

Our final model was trained on 4xA100 40GB using PyTorch's DDP. Training took 4.5 days.

Inference

We largely use nnU-Net’s inference infrastructure. The input series are dissected into a series of patches. Each patch gets blob regressed. We take the maximum per channel as class probability. We max-aggregate across patches for our final predictions.
We used the 2xT4 instances for the prediction and split the patches to predict for each input series evenly across the GPUs. We always use a single model, no ensemble. Inference takes approximately 8 hours.

Results

Unfortunately, we were hit quite hard by the instabilities of the Kaggle platform.
While our model finished training for 3000 epochs, only submissions until 1500 epochs (three days before the deadline) were successful (private and public LB 0.83). Subsequent submissions timed out, even though only the model weights differed.
Our internal validation showed that later checkpoints, TTA and Gaussian weighting of the patches would have probably further improved our performance (also previous submissions showed this).
Surprisingly, our internal performance went up until 0.9 which was not achieved on the leaderboard. We don’t

know where this shift comes from. One reason might be that we had to embed our inference in a try/catch block, else an error was thrown after 15min. We don't know exactly why.
We did not exploit the segmentation masks, which could have been added as auxiliary outputs during training to help the model localize better. Due to joining late, we didn't find the time to investigate this. However, other teams showed that this improved their performance.

What did not work?

- We also framed the problem as a detection problem, attempting to solve it with the self-configuration detection framework nnDetection. This approach performed better than the solution presented here in the public leaderboard, but it underperformed in the private leaderboard and in our internal validation.
    - nnDetection was trained on instance segmentation label versions, also on cropped data, consisting of a self-configured Retina U-Net architecture that learned from the aneurysm positions encoded as boxes and from the aneurysm segmentations . Unlike the solution here, it resampled the input series to an isometric space of [1.0, 1.0, 1.0] mm, training with a batch size of 4 for 100 epochs (2500 iterations per epoch), hybrid loss function combining L1 loss for the regression of box coordinates and focal loss for box class estimation, polynomial learning rate scheduling from an initial value of 0.001, and SGD optimizer with Nesterov momentum. Predicted boxes were postprocessed via non-maximum suppression with a 0.1 intersection over union threshold. We additionally managed to conduct inference with 8 test time augmentations and an inference patch overlap of 0.25.
- Isometric space resampling with [1.0, 1.0, 1.0] mm was also implemented, given its potential for faster image processing. However, it substantially worsened our results, so it was discontinued early on.
- We also explored co-training with external aneurysm datasets containing binary classes to better model the Aneurysm Present class (ADAM, Large IA Segmentation dataset, INSTED, Lausanne TOF-MRA Aneurysm Cohort, Royal Brisbane TOFMRA Intracranial Aneurysm Database, Jianxiaokuang aneurysm dataset. We realized afterwards that some of these datasets (ADAM, Large IA Segmentation dataset) were not allowed, so we discarded them and ran co-training without them. In the end, co-training did not really help, so we resorted back to training only on the challenge cases from scratch.
- We first started with processing the image as a whole but time limitations forced us to crop around the ROI which also resulted in better performance.
- As described above, in our final model we just max-aggregate the patch predictions. However, it is known that the model has some uncertainty close to the edges. A common strategy to mitigate this is gaussian weighting the predictions for each patch (high weight in the center, low weight and the borders). In earlier submissions we saw that this improved our performance. However, in our final model we could not apply this strategy because of platform instabilities.
- We also tried to train with larger patch sizes, which, surprisingly, did not contribute to improve our scores.

What would we have wished for?

We already stated in the discussion forum that the signature of the predict function was limiting us quite a lot in how we can parallelize processing. More about that here.
This requirement made it even more difficult for us since we required a specific numpy version which forced us to run our code as a subprocess. We ended up spawning a proxy worker with which we communicated via the std output. This added significant boilerplate and complexity, and felt unnatural.

We would have also wished for longer submission times, and a less dependent server architecture on the number of submissions sent by different teams, since many of our submissions timed out during the last few days of the challenge due to an increasing workload.
On a much broader scope: Kaggle's submission notebook style made it quite hard for us (also the last time). Having the possibility to just use Docker containers would have eased our lives a lot because they allow much higher flexibility.

Acknowledgements

We thank RSNA for organizing and Kaggle for hosting this competition. We furthermore want to give a shoutout to our Divisions of Medical Image Computing at the German Cancer Research Center, as well as Helmholtz Imaging

Supplemental Q&A

> **Ma Edward · 2025-10-16T20:19:15.397000**
>
> Thanks for sharing the informative writeup very much. Your team is really a symbol of elegant challenge solutions! It's impressive that a single model can achieve such high performance even if it is not fully optimized.
>
> It would be highly appreciated if you could share answers to the following questions.
>
> 1. How to automatically derive the [200, 160, 160] ROI?
> 2. How to use the images with AP for training? The corresponding mask will be full zero.
> 3. Is there any design to address the class imbalance?
> 4. TTA was not used in inference, right?
> 5. How different sphere size affect the performance based on the fold-0 split?
> 6. Do you have any plans to release the code? This task is much more challenging than the BYU task. Looking forward to diving into the implementation details.

> **Stefan Denner · 2025-10-16T20:50:33.427000**
>
> Hi @maedward,
> thanks for the kind words. :)
>
> 1. How to automatically derive the [200, 160, 160] ROI?
>   -> It is really as simple as we write it. We just take a center top crop of the image with the respective ROI size in mm.
> 2. How to use the images with AP for training? The corresponding mask will be full zero.
>   -> you mean without AP, right? Yes, the target heat map is zero-only. There is no issue with that.
> 3. Is there any design to address the class imbalance?
>   -> We thought of it. We also implemented class weighting and low prevalence class oversampling in our nnDetection efforts but there it did not help. For nnUNet we have not tried it out. But is probably worth a shot because we also noticed that one particular class was always achieving rather low performance.

However, we also noticed that some other classes which also had low prevalence were achieving quite reasonable performance so it is not only about the class prevalence.

4. TTA was not used in inference, right?
   -> Correct. With stepsize 0.5 and median spacing inference took to long with TTA.
5. How different sphere size affect the performance based on the fold-0 split?
   -> EDT25 0.888; EDT35 0.876, EDT45 0.893; EDT55 0.883; EDT65 0.896; EDT75 0.871; EDT85 0.871; EDT95 0.866
6. Do you have any plans to release the code? This task is much more challenging than the BYU task. Looking forward to diving into the implementation details
   -> Yes, we will release it next week :)